패킷 정의서 — DirectX12 TPS 4 인 개인전

클라이언트는 2 종, 서버는 8 종의 패킷을 보낸다. 아래 코드를 순서대로 이으면 공통 헤더 `Protocol.h` 하나가 되고, 클라이언트와 서버가 같은 파일을 쓴다.

1. 공통 규칙

모든 패킷은 3 바이트 헤더로 시작하는 구조체다. 헤더만 읽으면 어떤 패킷인지와 전체 크기를 알 수 있다.

- **헤더:** `size`(2 바이트, 헤더를 포함한 패킷 전체 바이트 수) + `type`(1 바이트, 패킷 종류).
- **패킹:** `#pragma pack(push, 1)`로 빈 바이트 없이 보낸다. 자료형은 크기가 고정된 `uint8_t`, `int8_t`, `uint16_t`, `uint32_t`, `float` 만 쓴다.
- **바이트 순서:** IP 와 포트만 네트워크 바이트 순서(`htons` 등)로 바꾼다. 게임 데이터는 참가 PC 가 모두 x64 Windows 인 것을 전제로 리틀 엔디안 그대로 보낸다.
- **TCP 보내기:** `SendPacket()` 하나로만 보낸다. 클라마다 있는 송신 임계 영역에 들어가서, `size` 바이트를 다 보낼 때까지 `send()`를 반복한다.
- **TCP 받기:** 헤더 3 바이트를 먼저 다 받고, `size - 3` 바이트를 마저 받는다(`recv(..., MSG_WAITALL)`).
- **UDP 받기:** `recvfrom()` 한 번이 패킷 하나다. 받은 바이트 수가 `header.size` 와 다르면 버린다.
- **목록 패킷(플레이어·적·총알 상태):** 배열은 최대 크기로 두고 앞에서 `count` 칸만 채워 보낸다. `size` = 배열 앞부분 8 바이트 + `count` × 칸 크기.
- **번호:** 입력 번호(`sequence`)와 틱 번호(`tick`)는 접속이 유지되는 동안 계속 늘고, 게임이 바뀌어도 0 으로 되돌리지 않는다. 되돌리면 두 번째 게임의 패킷이 전부 '늦게 온 것'으로 버려진다.
- **PlayerId:** 0~3. '없음'은 `INVALID_PLAYER_ID`(255).

2. 패킷 한눈에 보기

잊으면 안 되는 패킷은 TCP, 매 틱 새로 오는 패킷은 UDP 로 보낸다. 크기는 `static_assert` 로 확인한 값이다.

방향	패킷	방식	보내는 때	내용	크기(바이트)
클라 → 서버	`CS_Ready`	TCP	대기 버튼을 처음 누를 때 1 회	없음(헤더만)	3
클라 → 서버	`CS_Input`	UDP	매 프레임(30 회/초)	보낸 사람, 입력 번호, 이동, 카메라 yaw, 발사, 쉴드 누른 횟수	16
서버 → 클라	`SC_ConnectResult`	TCP	`accept` 직후 그 클라에게 1 회	승인 여부, 내 PlayerId, 거절 이유	6
서버 → 클라	`SC_LobbyState`	TCP · 전원	접속·준비·끊김·로비 복귀 때	접속 인원, 각자의 PlayerId 와 준비 여부	12
서버 → 클라	`SC_GameStart`	TCP · 전원	게임 준비 때 1 회	맵 번호, 참가 인원, 각자의 기체와 시작 위치	61
서버 → 클라	`SC_PlayerStates`	UDP · 클라마다	매 틱(30 회/초)	틱 번호, **나를 포함한 전원의** 위치·방향·체력·생존/유령·쉴드	8 + 20 × 인원 (최대 88)
서버 → 클라	`SC_EnemyStates`	UDP · 클라마다	매 틱	틱 번호, 살아 있는 적마다 번호·위치·방향	8 + 17 × 살아 있는 적 수 (최대

방향	패킷	방식	보내는 때	내용	크기(바이트)
					552)
서버 → 클라	`SC_BulletStates`	UDP · 클라마다	매 틱	틱 번호, 총알마다 위치	8 + 12 × 총알 수 (최대 776)
서버 → 클라	`SC_Event`	TCP · 전원	사망이 일어난 틱	종류, 죽은 쪽, 사망 위치	17
서버 → 클라	`SC_GameResult`	TCP · 전원	생존자가 1 명 이하가 된 틱, 1 회	승자 PlayerId	4

트래픽: Map1(적 15 명)에 총알 64 발이 모두 날아다니는 최악의 경우 틱당 88 + 263 + 776 = 1,127 바이트다. 30 회/1 초면 클라 1 명당 초당 약 34KB, 4 명이면 서버 송신이 초당 약 135KB 로 LAN 에서는 충분하다.

3. 상수와 enum

파일의 맨 앞부분이다. 포트 번호는 예시 값

```
#pragma once
#include <cstdint>

#pragma pack(push, 1)

//uint8_t  = 1 바이트
//uint16_t = 2 바이트
//uint32_t = 4 바이트

// ===== 상수 =====
constexpr uint8_t MAX_PLAYERS = 4;              // 최대 접속 인원
constexpr uint8_t MIN_PLAYERS_TO_START = 2;     // 게임을 시작할 수 있는 최소 인원
constexpr uint8_t MAX_ENEMIES = 32;             // 한 맵의 AI 적 최대 수
constexpr uint8_t MAX_BULLETS = 64;             // 동시에 날아다니는 총알 최대 수
constexpr uint8_t INVALID_PLAYER_ID = 255;      // "없음" (거절됨, 승자 없음)
constexpr int TICK_RATE = 30;                   // 서버 틱, 클라이언트 프레임 (회/초)
constexpr uint16_t SERVER_TCP_PORT = 9000;      // 서버 TCP 대기 포트 (예시 값)
constexpr uint16_t SERVER_UDP_PORT = 9001;      // 서버 UDP 입력 받기 포트 (예시 값)

// ===== enum =====
enum class EPacketType : uint8_t
{
    // 클라이언트 -> 서버
    CS_Ready = 1,
    CS_Input,

    // 서버 -> 클라이언트
    SC_ConnectResult = 100,
    SC_LobbyState,
    SC_GameStart,
    SC_PlayerStates,
    SC_EnemyStates,
    SC_BulletStates,
    SC_Event,
    SC_GameResult
};

enum class EPlayerType : uint8_t { None = 0, Helicopter, Tank };            // 기체
종류
enum class EPlayerState : uint8_t { Alive = 0, Ghost };                     // Ghost
= 사망 후 관전 (이동만 가능)
enum class EEventType : uint8_t { PlayerDied = 0, EnemyDied };              //
SC_Event 종류
enum class ERejectReason : uint8_t { None = 0, ServerFull, AlreadyStarted }; // 접속
거절 이유
```

```
// ===== 공통 헤더 =====
struct PacketHeader
{
    uint16_t size;          // 헤더를 포함한 패킷 전체 바이트 수
    EPacketType type;       // 패킷 종류
};
```

- `EPlayerType` 은 게임 코드에 같은 이름이 이미 있으면 여기서 빼고 기존 것을 쓴다. 값의 순서는 위와 같아야 한다.
- 게임 단계(로비·게임 중) 표시는 패킷으로 보내지 않는다. 서버 내부에만 두고, 클라는 받은 패킷으로 화면을 넘긴다.

4. 클라이언트 → 서버 구조체

클라이언트는 준비와 입력, 두 가지만 보낸다. 접속은 TCP 연결 자체가 요청이라 따로 요청 패킷이 없다.

```
// ===== 클라이언트 -> 서버 =====
struct CS_Ready             // TCP. 로비에서 대기 버튼을 처음 누를 때 1 회
{
    PacketHeader header;    // 내용 없음 (헤더만)
};

struct CS_Input             // UDP. 매 프레임 (30 회/초)
{
    PacketHeader header;
    uint8_t playerId;           // 보낸 플레이어 (UDP 는 연결이 없어서 패킷에 적음)
    uint32_t sequence;          // 입력 번호. 매 프레임 +1, 게임이 바뀌어도 되돌리지
않음
    int8_t moveForward;         // -1(S), 0, 1(W)
    int8_t moveRight;           // -1(A), 0, 1(D)
    float cameraYaw;            // 카메라 yaw (도). 이동 방향의 기준
    uint8_t fire;               // 좌클릭을 누르고 있으면 1
    uint8_t shieldPressCount;   // E 를 누른 횟수. 누를 때마다 +1, 서버는 값이 바뀌면
쉴드 토글
};
```

5. 서버 → 클라이언트 구조체

서버는 8 종을 보낸다. 매 틱 보내는 상태 3 종만 UDP 이고, 나머지는 모두 TCP 다.

5-1. 접속과 로비 (TCP)

```
// ===== 서버 -> 클라이언트 =====
struct SC_ConnectResult     // TCP. accept 직후 그 클라이언트에게 1 회
{
    PacketHeader header;
    uint8_t accepted;           // 1 = 승인, 0 = 거절
    uint8_t playerId;           // 내 PlayerId (거절이면 INVALID_PLAYER_ID)
    uint8_t rejectReason;       // ERejectReason
};

struct LobbyPlayerInfo
{
    uint8_t playerId;
    uint8_t ready;              // 대기 버튼을 눌렀으면 1
};

struct SC_LobbyState        // TCP. 접속·준비·끊김·로비 복귀 때 전원에게
{
    PacketHeader header;
    uint8_t playerCount;                        // 접속 인원 (앞에서 playerCount 칸이
유효)
    LobbyPlayerInfo players[MAX_PLAYERS];       // 항상 4 칸 모두 보냄
};
```

5-2. 게임 시작 (TCP)

```
struct PlayerStartInfo
{
    uint8_t playerId;
    uint8_t playerType;         // EPlayerType
    float x, y, z;              // 시작 위치
};

struct SC_GameStart         // TCP. 게임 준비 때 전원에게 1 회
{
    PacketHeader header;
    uint8_t mapNumber;                          // 1 ~ 3
    uint8_t playerCount;                        // 이번 게임 참가 인원
    PlayerStartInfo players[MAX_PLAYERS];       // 항상 4 칸 모두 보냄
};
```

5-3. 매 틱 상태 (UDP)

세 패킷 모두 앞부분이 헤더 + `tick` + `count` = 8 바이트로 같고, 배열은 `count` 칸만 보낸다.

```
struct PlayerStateInfo
{
    uint8_t playerId;
    uint8_t state;              // EPlayerState (Alive / Ghost)
    uint8_t shieldEnabled;      // 쉴드가 켜져 있으면 1
    uint8_t health;             // 0 ~ 100
    float x, y, z;              // 위치
    float yaw;                  // 기체가 바라보는 방향 (도)
};

struct SC_PlayerStates      // UDP. 매 틱, 클라마다 sendto
{
    PacketHeader header;
    uint32_t tick;                              // 틱 번호. 게임이 바뀌어도 되돌리지
않음
    uint8_t count;                              // 들어 있는 플레이어 수 (나 포함, 최대
4)
    PlayerStateInfo players[MAX_PLAYERS];       // 플레이어마다 1 칸. count 칸만 보냄
};

struct EnemyStateInfo
{
    uint8_t enemyIndex;         // 게임 시작 때 맵 Spawn 순서 (서버와 클라가 같음,
종류도 이 번호로 앎)
    float x, y, z;              // 위치
    float yaw;                  // 바라보는 방향 (도)
};

struct SC_EnemyStates       // UDP. 매 틱, 클라마다 sendto
{
    PacketHeader header;
    uint32_t tick;
    uint8_t count;                              // 살아 있는 적 수
    EnemyStateInfo enemies[MAX_ENEMIES];        // 살아 있는 적만 count 칸 보냄 (죽음은
SC_Event 로 알림)
};

struct BulletStateInfo
{
    float x, y, z;              // 총알 위치
};
```

```
struct SC_BulletStates      // UDP. 매 틱, 클라마다 sendto
{
    PacketHeader header;
    uint32_t tick;
    uint8_t count;                              // 날아가는 총알 수
    BulletStateInfo bullets[MAX_BULLETS];       // count 칸만 보냄
};
```

5-4. 이벤트와 결과 (TCP)

```
struct SC_Event             // TCP. 사망이 일어난 틱에 전원에게
{
    PacketHeader header;
    uint8_t eventType;          // EEventType
    uint8_t subjectId;          // PlayerDied: 죽은 PlayerId / EnemyDied: enemyIndex
    float x, y, z;              // 사망 위치 (폭발 연출용)
};

struct SC_GameResult        // TCP. 생존자가 1 명 이하가 된 틱에 전원에게 1 회
{
    PacketHeader header;
    uint8_t winnerPlayerId;     // 승자. 동시 사망으로 승자가 없으면 INVALID_PLAYER_ID
(전원 패배)
};

#pragma pack(pop)
```